

Homework – Print Characters in a Loop

This is a programming assignment that will be graded and scored as homework since it is so simple.

Write a program for the LC3. This program should be named ***hwPrintCharLoop.hex***. The file should be a proper hex file for the LC3 processor.

A proper hex file is a text file that contains only hex characters (0-9 and A-F).

Each line should be a four-digit hex number. Do not include a 0x or x at the beginning of this number.

The first line should be the starting address in memory, use 3000 to be safe.

The last line should be a halt (F025)

DO NOT add any extra spaces at the end of lines.

BE CAREFUL not to use the character O in place of the number 0 (zero) or the lower case L (l) in place of the 1 (one).

The lines between the first and last line should do the following:

R0 and R1 will contains ASCII codes for characters.

R0 is the start character and R1 contains the stop character.

Your program should print R0 and all subsequent characters until it reaches and prints the character as stored in R1.

Note that it is possible that R0 and R1 contain the same character in which case your program would print what is in R0 and see that it is done with only one character printed.

You can assume that $R0 \leq R1$. R1 will never be less than R0.

Do not initialize R0 and R1, simply assume the values have already been placed in these registers.

Write your program in assembly. Convert it manually to hex. Put the hex in hwPrintCharLoop.hex and upload.

Submit your hex file to Web-CAT. Make sure you or your text editor did not add **.txt** to the end of the filename.

Testing your program

1. Open you program in the simulator but do not run it.
2. Click on R0 and put the value x41 (ASCII A) in it.
3. Click on R1 and put the value x45 (ASCII E) in it.
4. Run your program.
5. You should see the following printed in the output section of the simulator.
ABCDE
6. Repeat the above with different values.

An algorithm

1. Print the character in R0
2. Check to see if $R1 == R0$
3. If the two are not equal increment R0 and go back to step 1
4. Done

INEFFICIENT CODE ISSUE

There is an efficient way to do this program, and an inefficient way. The test is designed to catch you if you do this inefficiently. You will only receive about an 80% grade.

The issue is that you are making the wrong thing negative.

R0 contains the character you are printing. This value increments every time you go through your loop.

R1 contains the character you are looking for. This value should not change.

The inefficiency is in the fact that $R0 - R1$ is different than $R1 - R0$. Either will work but one is better in this instance.

To make a number negative, you must invert it and add 1 to it which is 2 operations. If you make R0 negative, you must do it every time you go through the loop. If you make R1 negative, since it never changes, you only really must make it negative once.

If your loop runs 10 times, for example, if you invert R1 and do it before you go into the loop, you use 2 instructions for the inversion. But if you invert R0 instead, you are forced to do it every time the value changes, which means you must do it every time the program loops, which means 20 instructions executed.

If you ever have register that is constant and a register that changes, and you are comparing them repeatedly in a loop, always make the constant negative.

In general, your program should look like this:

```
    Make R1 negative
TOP
    OUT
    ADD R0, R0, 1
    Add R2, R1, R0
    BR (N?Z?P?) to TOP
    HALT
```